MATH4076: Exam Prep

Week 1: Floating Points and Errors

Iterative schemes: $x_{n+1} = f(x_n)$. In general a fixed point, x^* can be found when $x_{n+1} = x_n = x^*$ or any solutions to $x_n = f(x_n)$.
Errors: for an estimation x_n and the true value x, the errors are

- Absolute Error = $e_n = |x - x_n|$
- Relative Error = $e_n = \frac{|x-x_n|}{|x|}$ for $x \neq 0$

If $\lim_{n\to\infty} e_n = 0$ then we can say that the iterative method converges.
Numbers in Computers:

- Integers: binary digits, $\sum_{i=1}^{N} d_i * 2^{i-1}$.
- Floats: sign (S), exponent (E) and mantissa (M), $(-1)^S 2^{E-1023}(1.M)$

Week 2: Root Finding

Bisection Method

$$c = \frac{a+b}{2}$$

- Description: Seach from [a, b] using bisection algorithm.
- Conditions: f(a) and f(b) must be of opposite signs (for continuous f).
- Convergence: convergences linearly with rate of $\frac{1}{2}$.

Newton-Raphson Method

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- Description: Use root of the tangent line to approximate root of the function.
- Conditions: x_0 must be 'close' to the root.
- Convergence: convergence is quadratic.

Secant Method

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

- Description: Use estimation of f'(x) instead of a numeric derivative.
- Conditions: x_0 must be 'close' to the root.
- Convergence: convergence is quadratic.

Week 3: Systems of Equations

Multivariate Newton-Raphson Method

$$F(x) = F(x_n) + J_F(x - x_n)$$

- Description: Use root of the tangent line to approximate root of the function.
- Conditions: x_0 must be 'close' to the root.
- Convergence: convergence is quadratic.

Gauss-Jordan Elimination Find upper diagonal matrix, back substitution for true x.

- Description: Eliminate down then perform back propogation to find x.
- Conditions: Algebraic and will always reach the answer if possible.
- Convergence: $O(d^3)$ operations.

LU Factorisation Find A = LU then finding Ly = b and Ux = y.

- Description: Split the matrix A into L and U, which are lower/upper diagonal matrix.
- Conditions: Algebraic and will always reach the answer if possible.
- Convergence: $O(d^2)$ operations.

Week 4: Interpolation

Lagrange Interpolation

$$L_j(x) = \prod_{i \neq j} \frac{x - x_i}{x_j - x_i}$$

$$p_{n-1}(x) = \sum_{k=1}^{n} y_k L_k(x)$$

- Description: Find the unique n-1 interpolation polynomial.
- Conditions: n points must have unique x values.

Newtons Divided Difference Create divided differences table

$$\left[x_i ... x_j\right] = \frac{f\left[x_i ... x_{j-1}\right] - f\left[x_{i+1} ... x_j\right]}{x_i - x_j}$$

Using top row

$$p_{n-1}(x) = \sum_i f[x_1, ..., x_i] \prod_{j=1}^{i} (x - x_j)$$

- Description: Easier way of adding a new point.
- Conditions: provides the same polynomial as Lagrange polynomial.

Chebyshev's Interpolation

$$x_i = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{(2i-1)\pi}{2n}\right)$$

- Description: choose points to interpolate with lower error.
- Conditions: minimises error between x and each x_i.

Cubic Splines

$$s_i(x_i) = y_i$$

$$s'_{i-1}(x_i) = s'_{i(x_i)}$$

$$s''_{i-1}(x_i) = s''_i(x_i)$$

$$s''_1(x_1) = s''_{n-1}(x_n) = 0$$

$$s_i(x) = y_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i^3)$$

- Description: Solve a system of equations for the cubic splines interpolating each point.
- Conditions: Different boundary conditions for different behaviours at endpoints.

Week 5: Fourier Transform

Discrete Fourier Transform

$$F(x) = \sum_{j=0}^{m-1} x_j \exp\left(\frac{-2\pi i}{m} jk\right) = \sum_{j=0}^{m-1} x_j w_m^{jk}$$

- Description: Transforms data from x and y to frequency and amplitude. Filters and denoises data.
- Conditions: Removes white noise from periodic signals.

Week 6: Differentiation and Integration

Forward Difference

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{h}{2} f''(c)$$

- Description: Standard differentiation by first principles.
- Conditions: f must be differentiable.
- Convergence: O(h) first order method.

Centred Difference

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + \frac{h^2}{6} f''(c)$$

- Description: Derived by substracting the forward and backwards step.
- Conditions: f must be differentiable.
- Convergence: O(h^2) second order method.

Riemann Sums

$$\int_{x_0}^{x_n} \approx \sum_{i=0}^{n} f(x_0 + ih) * h$$

- Description: Standard integration by first principles.
- Conditions: f must be integrable.
- Convergence: O(h) first order method.

Trapeziod Rule

$$\frac{h}{2}(y_0 + y_1)$$

- Description: Calculate the area of a trapeziod between two points.
- Conditions: f must be integrable.
- Convergence: O(h^3) first order method. Composite reduces order to O(h^2).

Simpson's Rule

$$\frac{h}{3}(y_0 + 4y_1 + y_2)$$

- Description: Integrate between 3 points.
- Conditions: f must be integrable.
- Convergence: O(h^5) first order method. Composite reduces order to O(h^4).

Week 7: Numerical Integration

Adaptive Integration

$$A_{[a,b]} = \frac{h}{2}(f(a) + f(b))$$

- Description: Integrate in subintervals for different error values.
- Conditions: f must be integrable.
- Convergence: Error is O(h^k)) where k > 2.

Monte-Carlo Integration

$$\frac{1}{N}\sum_{(x,y)} 1_{\{f(x)<y\}}$$

- Description: Use random sampling of points to the integral area.
- Conditions: None, applicable to higher order and un-evaluatable functions.
- Convergence: O($h^{\frac{1}{2}}$), far worse than either trapezoid and simpsons.

Week 8: ODEs

Euler's method for IVP

$$t_{n+1} = t_n + h$$

$$y_{n+1} = y_n + h * \dot{y}(t_n, y_n)$$

- Description: using first principles definition of the derivative to solve next step
- Conditions: y is continuous and differentiable. For higher order equations, $\tilde{y}$ represents $(y, \dot{y}, ...,)$
- Convergence: O(h^2) method in a single step, error acculumates as we rely on past to move forward and is a O(h) method in aggregate.

Trapezoid method

$$y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_n + h, y_n + hf(t_n, y_n))]$$

- Description: Consider the average between current step and the euler next step.
- Conditions: Same as Euler's
- Convergence: O(h^3) method in a single step, error accumulates as we rely on past to move forward and is a O(h^2) method in aggregate.

Midpoint method

$$y_{n+1} = y_n + hf\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f(t_n, y_n)\right)$$

- Description: Consider the midpoint between the euler next step.
- Conditions: Same as Euler's
- Convergence: O(h^3) method in a single step, error accumulates as we rely on past to move forward and is a O(h^2) method in aggregate.

Runge-Kutta methods

$$y_{n+1} = y_n + \frac{h}{6}(s_1 + 2s_2 + 2s_3 + s_4)$$

$$s_i = f\left(t_n + \frac{h}{2}, y_n + hs_{i-1}\right), s_1 = f(t_n, y_n)$$

- Description: Each s_i is a different approximation of $\dot{y}$

Week 9: BVP for ODEs

Shooting method

$$\text{Starting at } \quad f'(t_a) = s, \text{IVP Solution is } y_{s(t)}$$

$$F(s) = y_{s(t_b)} - y_b$$

$$F(s^*) = 0, \text{ then } \quad y_{s^*}(t) \text{ is solution}$$

- Description: solve IVP with initial value as a variable, then back solve with the boundary condition.

Finite difference method

$$\begin{pmatrix} h^2-2 & 1 & 0 & 0 \\ 1 & h^2-2 & 1 & 0 \\ 0 & 1 & h^2-2 & 1 \\ 0 & 0 & 1 & h^2-2 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} = \begin{pmatrix} -y_a \\ 0 \\ 0 \\ -y_b \end{pmatrix}$$

- Description: Replace the derivatives with the finite differences. Then solving for other y values.

Collocation method

$$y(t) = \sum_{j=1}^{n} c_j \Phi_{j(t)}$$

$$\Phi_{j(t)} = t^{j-1}$$

- Description: reduce BVP to n algebraic equations in terms of c_j. Calculate derivatives using our estimated $y(t)$.

Week 10: PDEs

Finite difference method

$$f_{xx}(t,x) = \frac{f(t,x+h) - 2f(t,x) + f(t,x-h)}{h^2}$$

$$f_{t(t,x)} = \frac{f(t+k,x) - f(t,x)}{k}$$

$$A = \begin{pmatrix} 1-2\sigma & \sigma & 0 & 0 \\ \sigma & 1-2\sigma & \sigma & 0 \\ 0 & \sigma & 1-2\sigma & \sigma \\ 0 & 0 & \sigma & 1-2\sigma \end{pmatrix}, \delta_j = \begin{pmatrix} U_{0,j} \\ 0 \\ 0 \\ U_{N,j} \end{pmatrix}, U_{j+1} = AU_j + \delta_j$$

- Description: on a grid of $t_j = kj$ and $x_i = a + hi$ use equation and estimations to calculate the next step. Consider only j+1 as we step up in time.
- Conditions: the method is stable if $\sigma \leq \frac{1}{2}$ or $2Dk \leq h^2$.
- Convergence: method is $O(h^2) + O(k)$ method.

Backward difference method

$$A = \begin{pmatrix} 1+2\sigma & -\sigma & 0 & 0 \\ -\sigma & 1+2\sigma & -\sigma & 0 \\ 0 & -\sigma & 1+2\sigma & -\sigma \\ 0 & 0 & -\sigma & 1+2\sigma \end{pmatrix}$$

- Description: use the backward difference formula instead of forward.
- Conditions: Unlike the forward difference, this is ALWAYS stable!

Centred difference method

$$A = \begin{pmatrix} 2(1-\sigma^2) & \sigma^2 & 0 & 0 \\ \sigma^2 & 2(1-\sigma^2) & \sigma^2 & 0 \\ 0 & \sigma^2 & 2(1-\sigma^2) & \sigma^2 \\ 0 & 0 & \sigma^2 & 2(1-\sigma^2) \end{pmatrix}$$

- Description: use centred difference formula instead of forward.
- Conditions: method is stable if $\sigma \leq 1$ or $ck \leq h$

Week 11: Uncontrained Optimisation

- Convexity Condition: $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$. If $f \in C^2$ then f convex $\Leftrightarrow$ $\frac{\delta^2 f}{\delta x_i \delta x_j} \geq 0 \forall x_i, x_j$

Golden Search method Finding the minimum in an interval $x^* \in [a, b]$.
Choose $x_1, x_2 \in [a, b]$

$$x_1 \coloneqq a + (1-g)(b-a)$$

$$x_2 \coloneqq a + g(b-a)$$

- Description: Choose x_1, x_2 with the interval, then tighten the interval based on results.
- Conditions: Guaranteed to converge to a single minimum within interval.
- Convergence: Converges at a rate of g, where g is the golden ratio.

Successive Parabolic interpolation Find $p_2(x)$ with 3 points and use $x^* = \min p_2(x)$ and replace 1 of 3 points with x^*

- Description: Use the interpolating parabola to estimate the minimum of the interval
- Condition: Convergence not ever guaranteed.
- Convergence: faster than golden-search if converging

Naive Multidimensional Optimisation

- Description: fix all dimension then take steps in best downward direction, repeating until tolerance met for all dimensions.

Steepest Descent Find $v = \frac{-\nabla f(x_n)}{|\nabla f(x_n)|}$ Then minimise $f(x + sv), s \in \mathbb{R}$ $x_{n+1} = x_n - s^* v$

- Description: use the directional derivative to step towards direction of steepest descent.
- Conditions: Always converges to a minima.
- Convergence: Converges in linear time.

Week 12: Contrained Optimisation

Linear Problem $c^T x, Ax = b, x \geq 0$

- Description: series of linear equations, can be thought of as reducing down area on a space, then checking extrema.
- Condition: may have many solutions if any constraint is parallel to c.

Penalty methods For objective function $f(x)$, equality contraints $h_{i(x)}$, inequality contraints $g_{j(x)}$.

$$f_\sigma(x) = f(x) + \sum \sigma_i h_i^2(x) + \sum \sigma_j \left[\max\left(0, g_{j(x)}\right)\right]^2$$

- Description: apply a penalty for going outside of allowed region.

Simplex method

$$\sum a_i x_i \leq b \rightarrow \sum a_i x_i + s = b$$

- Description: add slack variables, then solve for all possible binary combinations of s_i
- Conditions: highly expensive as slack variables increase.

Gradient Descent

$$x' = x_n - \alpha \nabla f(x_n)$$

Accept if $f(x') < f(x_n)$ and constraints satisfied, reject otherwise.

Week 13: Applications

Markov Chain Monte Carlo Propose x' with a probability, then accept based on a probability $0 \leq A(x) \leq 1$

- Description: Have some probability to go in a non-optimal direction, to prevent getting stuck in local minima.

Simulated Annealing

$$T_{t+1} = \alpha T_t$$

- Description: Reduce "temperature" slowly, where temperture decides accept/reject.

Stochastic Gradient Descent

$$x_{t+1} = x_t - \alpha \nabla f(x_t), \alpha \geq 0$$

- Description: Sample the training dataset randomly, then use gradient descent formula in that direction.